학습 노트: asyncio 병렬 실행 패턴 (run_in_executor + gather)

작성일: 2026-05-24 관련 파일: `dev2/tools/python/pylint_tool.py`, `dev2/tools/python/bandit_tool.py`, `dev2/nodes/` (구현 예정: `run_tools` 노드) 복습 예정: D+1, D+3, D+7 관련 **Task**: Task 4-a — subprocess CLI 도구 병렬 실행 노드 구현

한 줄 요약

이미 만들어 둔 동기(sync) 검사 함수(`run_pylint`, `run_bandit`)를 고치지 않고, 별도의 작업자 스레드에 던져서 여러 개를 "동시에" 굴리고, 30초 안에 끝난 것만 모아오는 방법.

들어가기 전: 이 노트와 **`subprocess_async.md`**의 차이 (중요)

같은 "병렬 실행"이지만 두 가지 길이 있습니다. 둘을 헷갈리면 안 됩니다.

구분	`subprocess_async.md` (Task 3)	이 노트 **`asyncio_parallel.md` (Task 4-a)**
접근법	도구 실행을 처음부터 `async`로 다시 작성	기존 동기 함수를 그대로 두고 비동기로 "감싼다"
핵심 함수	`asyncio.create_subprocess_exec`	**`loop.run_in_executor`**
함수 모양	`async def run_cli(...)`	`def run_pylint(...)` (기존 그대로)
언제	새로 만들 때, 가장 효율적	이미 `subprocess.run`으로 짜둔 코드를 병렬화할 때

> 우리 상황: `dev2/tools/python/pylint_tool.py`의 `run_pylint`는 이미 동기 **`subprocess.run`**으로 완성돼 있습니다. 이걸 다시 안 짜고 그대로 살리려면 `run_in_executor`가 정답입니다. 그래서 Task 4-a는 이 노트의 패턴을 씁니다.

개념 1. asyncio 이벤트 루프와 coroutine (기초)

🍕 비유

식당에 종업원이 한 명(이벤트 루프) 있습니다. 손님이 "물 주세요" 하면 종업원이 주방에 주문만 넣고, 물이 나오는 동안 멍하니 서 있지 않고 다른 테이블 주문을 받으러 갑니다. 물이 다 되면 그때 가서 가져다줍니다. 종업원은 한 명인데도 여러 테이블이 동시에 굴러갑니다.

여기서 "물 좀 가져다줘"처럼 잠깐 멈췄다**(await)** 나중에 이어서 할 수 있는 작업이 coroutine입니다.

📚 개념

- 이벤트 루프**(event loop)**: 비동기 작업들을 번갈아 실행하는 "단 하나의 관리자". asyncio 프로그램의 심장.
- **coroutine**(코루틴): `async def`로 정의한 함수. 중간에 `await`를 만나면 멈췄다가, 기다리던 일이 끝나면 그 자리부터 다시 이어 실행된다.
- **await**: "이건 시간이 걸리니, 끝날 때까지 나는 잠깐 양보할게"라는 표시. 이때 이벤트 루프는 다른 coroutine을 실행한다.

핵심 원리: 비동기는 **CPU**가 여러 개라서 빠른 게 아니라, "기다리는 시간(I/O 대기)"에 다른 일을 끼워 넣어서 전체 시간을 줄이는 것입니다. (스레드/멀티코어와 다른 개념 — 개념 6에서 다시)

🤔 왜 쓰나

- 없으면: pylint가 디스크에서 파일을 읽고 분석하는 동안 내 프로그램은 그냥 멈춰서 기다림 → 그 시간이 통째로 낭비
- 있으면: 그 대기 시간에 bandit, radon을 같이 시작 → 전체 시간이 "가장 느린 한 개"만큼으로 줄어듦

개념 2. asyncio.gather() — 여러 coroutine을 동시 실행

🍕 비유

배달 앱으로 짜장면(10분)과 탕수육(8분)을 동시에 주문하면 18분이 아니라 10분 만에 다 옵니다. `gather`는 "여러 주문을 한꺼번에 넣고, 전부 도착할 때까지 기다린 뒤, 받은 음식을 순서대로 정리해서 주는" 역할입니다.

📚 개념

`asyncio.gather(*coros)`는 넘겨준 여러 coroutine을 동시에 시작하고, 전부 끝날 때까지 기다린 뒤 결과를 넘긴 순서 그대로 리스트로 반환한다.

```
results = await asyncio.gather(coro_a, coro_b, coro_c)
# results[0]은 coro_a 결과, results[1]은 coro_b 결과 ... (완료 순서가 아니라 넘긴 순서)
```

🧐 왜 쓰나

- 없으면: pylint(30초) 끝나고 → bandit(30초) → radon(20초)... 순차로 = 80초. CLAUDE.md §6의 "에이전트 30초 타임아웃"을 넘겨서 실패
- 있으면: 셋을 동시에 → 가장 느린 30초만 기다림 → 타임아웃 안에 들어옴

개념 3. loop.run_in_executor() — 동기 함수를 비동기로 감싸기 ⭐ 핵심

🍕 비유

종업원(이벤트 루프)은 한 명뿐인데, `run_pylint`는 "끝날 때까지 자리를 안 비켜주는 고집 센 손님"입니다. 동기 `subprocess.run`은 결과가 나올 때까지 종업원을 30초간 붙잡아 둡니다. 그러면 종업원이 다른 테이블(다른 도구)을 못 받으니 "동시 실행"이 깨집니다.

해결책: 뒤편에 아르바이트생(작업자 스레드)들을 두고, 고집 센 손님은 알바에게 떠넘깁니다. 종업원은 "알바한테 맡겼으니 끝나면 알려줘" 하고 곧장 다른 테이블로 갑니다. 알바가 일하는 동안 종업원은 자유롭습니다.

`run_in_executor`가 바로 "동기 함수를 알바에게 떠넘기는" 도구입니다.

📚 개념

`loop.run_in_executor(executor, func, *args)`는 블로킹(자리를 안 비키는) 동기 함수를 별도의 스레드/프로세스(executor)에서 실행하고, 그 실행을 **await** 가능한 **coroutine**처럼 만들어준다.

- 첫 번째 인자 `executor`에 `None`을 주면 → 파이썬의 기본 **ThreadPoolExecutor** 사용
- `func`는 `()` 없이 함수 이름만 (호출이 아니라 "넘겨주기")
- 인자는 그 뒤에 나열: `run_in_executor(None, run_pylint, py_files)`

왜 이게 필요한가 (핵심 이유): > `subprocess.run`은 **blocking**(블로킹) 함수다. 즉 결과가 나올 때까지 이벤트 루프를 통째로 멈춘다. `gather`에 동기 함수를 그냥 넣으면 "동시"가 되지

않고 한 줄로 서버린다. `run_in_executor`로 다른 스레드에 보내야 이벤트 루프가 자유로워지고 진짜 병렬이 된다.

🧐 왜 쓰나

- 없으면: 기존 `run_pylint`, `run_bandit`을 전부 `async def` + `create_subprocess_exec`로 다시 짜야 함 → 이미 잘 동작하는 동기 함수 10개를 재작성하는 헛수고
- 있으면: `pylint_tool.py`, `bandit_tool.py`의 동기 함수를 한 줄도 안 고치고 그대로 병렬 실행 가능

```
# 이게 안 되는 이유 (gather에 동기 함수 결과를 그냥 넣음)
results = await asyncio.gather(run_pylint(files))  # run_pylint(files)가 먼저 끝까지 실행됨 → 동시성 깨짐, 타입도 안 맞음

# 올바른 방법 (동기 함수를 executor에 떠넘김)
loop = asyncio.get_event_loop()
results = await asyncio.gather(
    loop.run_in_executor(None, run_pylint, files),   # 알바 1에게
    loop.run_in_executor(None, run_bandit, files),   # 알바 2에게
)  # 두 알바가 동시에 일함 → 진짜 병렬
```

개념 4. asyncio.wait_for() — 타임아웃 적용 (CLAUDE.md §6 필수)

🍕 비유

요리사에게 "30분 안에 안 나오면 주문 취소할게"라고 시간 제한을 거는 것. 한 도구가 무한 루프에 빠지거나 거대한 파일로 영원히 안 끝나도, 30초가 지나면 강제로 끊고 넘어갑니다.

📚 개념

> `asyncio.wait_for(coro, timeout=30.0)`은 coroutine에 제한 시간을 씌운다. 시간 안에 끝나면 정상 결과를, 시간을 넘기면 `asyncio.TimeoutError`를 발생시킨다.

🧐 왜 쓰나

- 없으면: 도구 하나가 멈추면 전체 PR 리뷰가 영원히 끝나지 않음. CLAUDE.md §6은 "모든 분석 에이전트 30초 타임아웃"을 강제
- 있으면: 30초 넘은 도구만 잘라내고 나머지 결과로 진행

> ⚠️ run_in_executor의 함정: `wait_for`가 30초 후 `TimeoutError`를 내도, 이미 작업자 스레드에서 돌고 있는 **`subprocess.run`**은 자동으로 안 멈춥니다. 파이썬은 실행 중인 스레드를 강제로 죽일 수 없기 때문입니다. 그래서 동기 함수 안의 **`subprocess.run`**에도

`timeout=30`을 주는 2중 안전장치가 필요합니다. (실제로 `pylint_tool.py` 40~43줄을 보면 `subprocess.run(..., timeout=30)`이 이미 들어 있습니다.)

개념 5. return_exceptions=True — 하나 실패해도 나머지 수집

🍕 비유

5개 음식을 동시 주문했는데 탕수육 가게가 문을 닫았습니다. `return_exceptions=False`(기본값)면 "주문 전체 취소"가 됩니다. `True`면 "탕수육은 '품절' 메모로 받고, 나머지 4개는 정상 수령"합니다.

🛍️ 개념

`asyncio.gather(..., return_exceptions=True)`를 주면, 어떤 coroutine이 예외를 던져도 `gather` 전체가 중단되지 않는다. 그 자리에는 예외 객체 자체가 결과로 담겨서 돌아온다.

```
results = await asyncio.gather(*tasks, return_exceptions=True)
# 예: [ [pylint이슈들], TimeoutError(...), [bandit이슈들] ]
#         성공              실패(예외 객체)      성공
```

🧐 왜 쓰나

- 없으면: bandit이 미설치라 에러 1번 나면 pylint·radon 결과까지 전부 날아감 → 부분 결과조차 못 줌
- 있으면: 실패한 도구는 건너뛰고(`isinstance(r, Exception)`로 걸러냄) 나머지 이슈는 정상 보고 → CLAUDE.md §12 "도구 실패 시 빈 결과로 진행" 정책과 일치

개념 6. ThreadPoolExecutor vs ProcessPoolExecutor — 무엇을 쓰나

🍕 비유

- **ThreadPoolExecutor(스레드)**: 한 사무실 안의 여러 직원. 같은 책상(메모리)을 공유. 전화 받기/택배 기다리기처럼 기다리는 일**(I/O)** 에 강함.
- **ProcessPoolExecutor(프로세스)**: 아예 다른 사무실(독립 메모리). 무거운 계산을 진짜 동시에 여러 CPU로 돌릴 때 강함.

🧤 개념

구분	ThreadPoolExecutor	ProcessPoolExecutor
단위	스레드 (메모리 공유)	프로세스 (메모리 분리)
강한 작업	**I/O** 대기 (파일/네트워크/외부 명령 대기)	**CPU** 계산 (순수 파이썬 무거운 연산)
GIL 영향	받음 (파이썬 계산은 동시 안 됨)	안 받음 (진짜 병렬 계산)
비용	가벼움	무거움 (프로세스 생성·데이터 복사)

> 핵심: 파이썬에는 **GIL(Global Interpreter Lock)** 이 있어서 스레드로는 순수 파이썬 계산을 동시에 못 한다. 하지만 `subprocess.run`은 다르다.

🧐 우리는 어느 것? → **ThreadPoolExecutor (즉 `run_in_executor(None, ...)`)**

이유: > `subprocess.run`은 "외부 프로그램**(pylint.exe** 등)이 일하는 동안 그냥 기다리는" I/O 대기 작업이다. 실제 분석 계산은 별도 프로세스인 pylint가 하지, 내 파이썬이 하지 않는다. 따라서 GIL과 무관하고, 가벼운 ThreadPoolExecutor로 충분히 병렬이 된다. ProcessPoolExecutor는 불필요하게 무겁고, 함수·인자를 프로세스 간 복사(pickle)해야 하는 제약까지 생긴다.

`run_in_executor`의 첫 인자에 `None`을 주는 것 = "기본 ThreadPoolExecutor 써라"라는 뜻입니다. 그래서 우리 코드 패턴이 `loop.run_in_executor(None, run_pylint, py_files)`인 것입니다.

💻 실제 코드 패턴 분석 (Task 4-a 구현 형태)

```
async def run_tools_parallel(files: list[str]) -> list[dict]:
    loop = asyncio.get_event_loop()                    # ① 이벤트 루프 가져오기
    py_files = [f for f in files if f.endswith(".py")]  # 라우터로 .py만 추림

    tasks = [
        asyncio.wait_for(                              # ④ 30초 타임아웃 씌우기
            loop.run_in_executor(None, run_pylint, py_files),  # ③ 동기함수→스레드, None=ThreadPool
            timeout=30.0,
        ),
        asyncio.wait_for(
            loop.run_in_executor(None, run_bandit, py_files),
            timeout=30.0,
        ),
    ]
    results = await asyncio.gather(*tasks, return_exceptions=True)  # ②⑤ 동시실행+실패허용
```

```
    issues = []
    for r in results:
        if isinstance(r, Exception):                    # ⑤ 타임아웃/실패한 도구는 건너
뜀
            continue
        issues.extend(r)                                # 성공한 도구 이슈만 누적
    return issues
```

단계별 흐름

1. **`asyncio.get_event_loop()`**: 현재 돌고 있는 이벤트 루프(종업원)를 가져온다. `run_in_executor`를 호출하려면 루프가 필요.
2. **`loop.run_in_executor(None, run_pylint, py_files)`** (개념 3): 동기 함수 `run_pylint`를 기본 ThreadPoolExecutor에 떠넘긴다. `run_pylint(py_files)`처럼 호출하지 않고 함수와 인자를 따로 넘기는 점에 주의. 반환값은 await 가능한 객체.
3. **`asyncio.wait_for(..., timeout=30.0)`** (개념 4): 위 작업에 30초 제한을 씌운다. CLAUDE.md §6 준수.
4. **`asyncio.gather(*tasks, return_exceptions=True)`** (개념 2·5): 두 작업을 동시에 시작하고 둘 다 끝날 때까지 await. 하나가 실패해도 예외 객체로 받아 전체는 살린다.
5. 결과 정리 루프: `isinstance(r, Exception)`로 실패/타임아웃 도구를 걸러내고, 성공한 도구의 이슈만 `extend`로 모은다.

코드 학습 리뷰 (개선 기회)

- 잘 쓴 패턴: `return_exceptions=True` + `isinstance` 필터 → 도구 하나가 죽어도 부분 결과 보장. CLAUDE.md §12와 정확히 일치.
- 개선 포인트 **1**: `asyncio.get_event_loop()`는 파이썬 3.10+에서 비권장 경고가 날 수 있다. coroutine 안에서는 **`asyncio.get_running_loop()`** 가 더 안전하다. → 이를 "현재 실행 중인 루프만 명시적으로 가져오기"라고 한다.
- 개선 포인트 **2**: 실패를 `continue`로 조용히 넘기면 "왜 bandit 결과가 없지?"를 나중에 알 수 없다. 건너뛰기 전에 `logger.warning(f"tool failed: {r!r}")` 한 줄을 남기면 CLAUDE.md §10-④("에러는 삼키지 않는다") 정신에 더 부합한다.
- 개선 포인트 **3** (가독성): 더 간결한 최신 문법은 `asyncio.to_thread(run_pylint, py_files)` 다. 이건 `loop.run_in_executor(None, run_pylint, py_files)`와 같은 일을 한 줄로 한다 (파이썬 3.9+). 다만 executor를 직접 고를 수 없으므로, 기본 ThreadPool로 충분한 지금 상황엔 잘 맞는다.

3rd-Project-sub2 프로젝트에서

파일: - `dev2/tools/python/pylint_tool.py` → `run_pylint(files)` (동기, 그대로 재사용) - `dev2/tools/python/bandit_tool.py` → `run_bandit(files)` (동기, 그대로 재사용) - `dev2/`

`tools/router.py` → `detect_languages()`로 파일을 언어별로 그룹핑 (어떤 도구를 돌릴지 결정) - `dev2/nodes/` → 이번 **Task 4-a**에서 만들 **`run_tools`** 노드가 위 패턴으로 도구들을 병렬 실행

역할 흐름: 1. 라우터(`router.py`)가 PR diff의 파일을 언어별로 나눈다 (.py는 pylint+radon+bandit+semgrep). 2. `run_tools` 노드가 각 동기 도구 함수를 `run_in_executor`로 감싸 `gather`로 동시 실행한다. 3. 30초 타임아웃을 넘기거나 실패한 도구는 건너뛰고, 성공한 이슈만 모은다. 4. 모은 이슈를 Output Contract A(`schema.py`의 `AgentResult`, CLAUDE.md §5-A)로 변환해 Dev4에게 전달한다.

> CLAUDE.md §6 연결: 타임아웃 처리(`try/except asyncio.TimeoutError`)는 `async with AgentHarness()` 바깥에 둬야 `agent_traces.status`가 잘못 'success'로 찍히지 않는다. 이번 패턴은 `wait_for` + `return_exceptions=True`로 예외를 gather 결과 리스트에서 받으므로, harness 블록 안에서 예외를 삼키지 않도록 노드 구조를 짤 때 주의.

연관 개념

- **subprocess + create_subprocess_exec** → 도구를 처음부터 async로 짜는 다른 길 (`docs/learning/ai/subprocess_async.md`). 우리는 기존 동기 함수 재사용이라 run_in_executor 채택
- **asyncio.gather** → 동시 실행 + 결과 모으기 (`docs/learning/python/asyncio_gather.md`)
- **GIL (Global Interpreter Lock)** → 파이썬이 한 번에 한 스레드만 파이썬 코드를 실행하게 하는 잠금. subprocess는 외부 프로세스라 GIL과 무관해서 ThreadPool로 충분
- **asyncio.to_thread** → `run_in_executor(None, ...)`의 최신 단축 문법 (파이썬 3.9+)
- 좀비/미정리 작업 → run_in_executor의 스레드는 wait_for가 끊겨도 안 멈추므로 subprocess.run 자체의 timeout이 2중 안전장치

복습 퀴즈 (3문항)

1. **(이해)** `asyncio.gather`에 동기 함수 `run_pylint(files)`를 그냥 넣으면 왜 진짜 병렬이 되지 않는가? 이를 해결하려면 어떤 함수로 감싸야 하는가?
   - 답: `subprocess.run`이 들어있는 동기 함수는 blocking이라 이벤트 루프(종업원)를 끝까지 붙잡아 동시 실행을 막는다. `loop.run_in_executor(None, run_pylint, files)`로 작업자 스레드에 떠넘겨야 루프가 자유로워져 진짜 병렬이 된다.
2. **(적용)** `loop.run_in_executor(None, run_bandit, py_files)`에서 (a) 첫 인자 `None`은 무슨 뜻이며, (b) `run_bandit` 뒤에 `()`를 붙이지 않고 인자를 따로 넘기는 이유는?

- 답: (a) `None`은 "파이썬 기본 ThreadPoolExecutor를 써라"는 뜻이다. (b) executor는 함수를 "나중에 자기가 호출"해야 하므로, 즉시 실행되는 `run_bandit(py_files)` 호출 결과가 아니라 함수 객체 `run_bandit`와 인자 `py_files`를 분리해서 넘긴다.

3. **(**비교**)** subprocess로 외부 CLI 도구를 병렬 실행할 때 ThreadPoolExecutor와 ProcessPoolExecutor 중 무엇이 적합하며 그 이유는? GIL은 왜 문제가 안 되는가?
   - 답: ThreadPoolExecutor가 적합하다. subprocess.run은 외부 프로그램이 일하는 동안 기다리는 I/O 대기 작업이고, 실제 계산은 내 파이썬이 아니라 별도 프로세스(pylint 등)가 한다. 따라서 파이썬 코드를 동시에 못 돌리게 막는 GIL의 영향을 받지 않아, 가볍고 빠른 ThreadPool로 충분하다. ProcessPool은 불필요하게 무겁고 pickle 제약까지 생긴다.